

The service runs on peers – not on the application – and uses the network metadata information maintained by the gossip communication layer to find out which peers are online. It also fetches information such as any relevant endorsement policies from the peer's state database. With service discovery, applications no longer need to specify which peers they need endorsements from. The SDK can simply send a query to the discovery service, asking which peers are needed, given a channel and a chaincode ID.

## Capabilities of the discovery service

The discovery service can respond to the following queries.

**Configuration query:** Returns the MSPConfig of all organisations in the channel along with the orderer endpoints of the channel.

**Peer membership query:** Returns the peers that have joined the channel.

**Endorsement query:** Returns an endorsement descriptor for given chaincode(s) in a channel.

**Local peer membership query:** Returns the local membership information of the peer that responds to the query. By default, the client needs to be an administrator for the peer to respond to this query.

## How it works

The discovery service will then compute a descriptor comprising two objects.

**Layouts:** This is a list of groups of peers, and a corresponding number of peers should be selected from each group.

**Group-to-peer mapping:** This is from the groups in the layouts to the peers of the channel. In practice, each group will most likely comprise peers that represent individual organisations, but because the service API is generic and ignorant about organisations, this is just a *group*.

The following is an example of a descriptor from the evaluation of a policy of AND(Org1, Org2) where there are two peers in each of the organisations:

```
Layouts: [
  QuantitiesByGroup: {
    "Org1": 1,
    "Org2": 1,
  },
  EndorsersByGroups: {
    "Org1": [peer0.org1, peer1.org1],
    "Org2": [peer0.org2, peer1.org2]
  }
]
```

In other words, the endorsement policy requires a signature from one peer in Org1 and one peer in Org2. And it provides the names of available peers in those orgs who can endorse (peer0 and peer1 in both Org1 and in Org2).

The SDK then selects a random layout from the list. In the example above, the endorsement policy is Org1 AND Org2. If instead it was an OR policy, the SDK would randomly select either Org1 or Org2, since a signature from a peer from either Org would satisfy the policy.

After the SDK has selected a layout, it selects from the peers in the layout based on criteria specified on the client side (the SDK can do this because it has access to metadata like ledger height). For example, it can prefer peers with higher ledger heights over others – or to exclude peers whose applications have been discovered to be offline – according to the number of peers from each group in the layout. If no single peer is preferable based on the criteria, the SDK will randomly select from the peers that best meet the criteria.

**Note:** When the peer is running with TLS enabled, the client must provide a TLS certificate when connecting to the peer. If the peer isn't configured to verify client certificates (clientAuthRequired is false), this TLS certificate can be self-signed.

END 

By: Swapnil Kulkarni

The author is an open source enthusiast with experience in blockchain, Cloud Native solutions, containers and enterprise software product architectures. He has diverse experience with different private and hybrid cloud architectures, e.g., Amazon Web Services, Azure, OpenStack, CloudStack, IBM Cloud, etc. He is a technology hobbyist and writes at [cloudnativetech.wordpress.com](https://cloudnativetech.wordpress.com).

www.efy.in

**EFYGROUP**  
Technology Drives Us

**“Nothing except the mint can make money without advertising.”**



-Thomas Babington Macaulay

**OpenSource**  
THE COMPLETE MAGAZINE ON OPEN SOURCE

**CLOUD COMPUTING**

1720 pages